UNIDAD 7. EXCEPCIONES Y COLECCIONES

Índice

1. Fundamentos de las excepciones 2
   1.1. Concepto de Excepción: ¿Qué ocurre cuando algo sale mal? 2
   1.2. Jerarquía de Excepciones en Java 3
2. Manejo y Control de Excepciones 6
   2.1. Captura de Excepciones: El bloque `try-catch-finally` 6
   2.2. Propagación: Uso de `throws` para delegar la responsabilidad 9
   2.3. Lanzamiento de Excepciones: Uso de `throw` 11
   2.4. Creación de Excepciones Personalizadas 13
3. El Java Collections Framework (JCF) 16
   3.1. Tipos de Colecciones y Estructuras 16
   3.2. Jerarquía de la Interfaz Collection 19
4. Operaciones y Tipado 22
   4.1. Operaciones con Colecciones 22
   4.2. Uso de Clases y Métodos Genéricos en Colecciones 25
Resumen 27
   Manejo de Excepciones (Haciendo el código robusto) 27
   Java Collections Framework (Gestión eficiente de datos) 28

1. Fundamentos de las excepciones

1.1. Concepto de Excepción: ¿Qué ocurre cuando algo sale mal?

Una **excepción** es un evento que ocurre durante la ejecución de un programa y que **interrumpe el flujo normal de las instrucciones**.

Imagina que tu código es una receta de cocina. El flujo normal es: cortar, mezclar, hornear. Una excepción sería "se acabó el gas del horno". No puedes seguir con el siguiente paso (hornear) de la manera habitual. Si no tienes un plan alternativo (un manejo de excepción), el proceso se detiene abruptamente y la cena se arruina (el programa colapsa o "crashea").

Definición clave: Una excepción es la forma que tiene el lenguaje (como Java) de decirte: *"No puedo continuar haciendo lo que me pediste debido a una situación inesperada. Aquí tienes un objeto con los detalles del problema; decide tú qué hacer"*.

Diferencia entre los tres grandes tipos de errores

Para entender dónde encajan las excepciones, es vital distinguir entre los tres "villanos" de la programación.

1. Error de Compilación (Syntax Error)

Ocurre **antes** de que el programa se ejecute. El compilador actúa como un profesor de gramática estricto; si no sigues las reglas del lenguaje, no te deja continuar.

- **Causa:** Escribir mal una palabra reservada, olvidar un punto y coma, o intentar guardar texto en una variable numérica.
- **Resultado:** No se genera el archivo ejecutable (`.class` en Java). El programa **nunca llega a correr**.
- **Detección:** El IDE (como Eclipse o IntelliJ) te lo marca en rojo inmediatamente.

Ejemplo:
Java
```
int numero = "Hola"; // Error: No puedes guardar un String en un
int
```

2. Error de Lógica (Logic Error / Semantic Error)

El programa compila y se ejecuta sin "crashear", pero **hace lo incorrecto**. Son los errores más difíciles de detectar porque el ordenador hace exactamente lo que le dijiste, pero tú le dijiste algo equivocado.

- **Causa:** Fórmulas matemáticas incorrectas, bucles infinitos por mal diseño, o algoritmos mal planteados.
- **Resultado:** El programa termina, pero el resultado es erróneo.
- **Detección:** Requiere pruebas unitarias (testing) y revisión manual del código.

Ejemplo:
Java
```
// Queremos calcular el promedio de 5 y 5 (debería ser 5)
double promedio = 5 + 5 / 2;
```

```
// Resultado: 7.5 (Error de lógica por falta de paréntesis)
// El ordenador obedeció la jerarquía de operaciones, pero no era
lo que queríamos.
```

3. Error en Tiempo de Ejecución (Runtime Error)

Aquí es donde viven las **Excepciones**. El código está bien escrito (sin errores de compilación) y la lógica parece correcta, pero durante la ejecución ocurre algo inesperado que impide continuar.

- **Causa:** Datos de entrada inválidos, recursos externos no disponibles, división por cero.
- **Resultado:** El programa se detiene abruptamente (crashea) y muestra una "traza de pila" (stack trace) en la consola, a menos que capturemos el error.
- **Detección:** Ocurre mientras el usuario usa la aplicación.

Ejemplo:
Java
```
int a = 10;
int b = 0;
int resultado = a / b; // ¡CRASH! ArithmeticException: / by zero
```

Resumen Comparativo

Tipo de Error	¿Cuándo ocurre?	¿Quién lo detecta?	¿Se ejecuta el programa?	Ejemplo clásico
Compilación	Al escribir/compilar	El Compilador (javac)	**No**	Falta un `;`
Lógica	Al usar el programa	El Usuario / Tester	**Sí** (pero da datos malos)	Sumar en vez de restar
Tiempo de Ejecución	Al usar el programa	La JVM (Java Virtual Machine)	**Sí** (hasta que falla)	División por cero, `NullPointerException`

Nos centraremos exclusivamente en los Errores en Tiempo de Ejecución, aprendiendo a usar bloques `try-catch` para evitar que el programa se cierre cuando estos ocurren.

1.2. Jerarquía de Excepciones en Java

En Java, las excepciones no son conceptos abstractos; son **Objetos**. Y como buenos objetos, están organizados en una jerarquía de clases (herencia). Todo problema que pueda ser "lanzado" (`thrown`) en Java desciende de una clase madre suprema.

1. La clase `Throwable`

En la cima de la pirámide se encuentra la clase `java.lang.Throwable`.

- **Función:** Es la superclase de todos los errores y excepciones.
- **Regla de oro:** Solo los objetos que son instancias de esta clase (o de sus hijas) pueden ser lanzados por la JVM o por la instrucción `throw`.

Justo debajo de `Throwable`, la jerarquía se divide en dos caminos críticos:

2. Error vs. Exception (La gran división)

Aquí distinguimos entre "catástrofes del sistema" y "problemas del programa".

A. La clase `Error` (Irrecuperable)

Representa problemas graves a nivel de la Máquina Virtual de Java (JVM) o del hardware.

- **Naturaleza:** Son errores fatales fuera del control de tu aplicación.
- **¿Qué hacer? Nada.** No debes intentar capturarlos (`catch`). Si ocurre un `Error`, lo más sano es que el programa termine.
- **Ejemplos:**
  - `OutOfMemoryError`: La JVM se quedó sin memoria RAM.
  - `StackOverflowError`: Desbordamiento de pila (comúnmente por una recursividad infinita).

Analogía: Un `Error` es como si tu edificio se quedara sin electricidad o hubiera un terremoto. No puedes arreglar eso reorganizando los muebles de tu oficina.

B. La clase `Exception` (Recuperable)

Representa condiciones que una aplicación razonable podría querer capturar y tratar.

- **Naturaleza:** Problemas que ocurren durante la ejecución del programa pero que no son fatales para el sistema operativo o la JVM.
- **¿Qué hacer?** Aquí es donde aplicamos nuestro código de manejo de excepciones para evitar que el programa se cierre.
- **Ejemplos:** Intentar leer un archivo que no existe, error de conexión a base de datos.

3. Checked Exceptions vs. Unchecked Exceptions

Dentro de la rama de `Exception`, existe una subdivisión que suele confundir a los programadores principiantes pero que define cómo escribimos código en Java.

A. Unchecked Exceptions (Excepciones No Verificadas)

Son aquellas que **heredan de `RuntimeException`**.

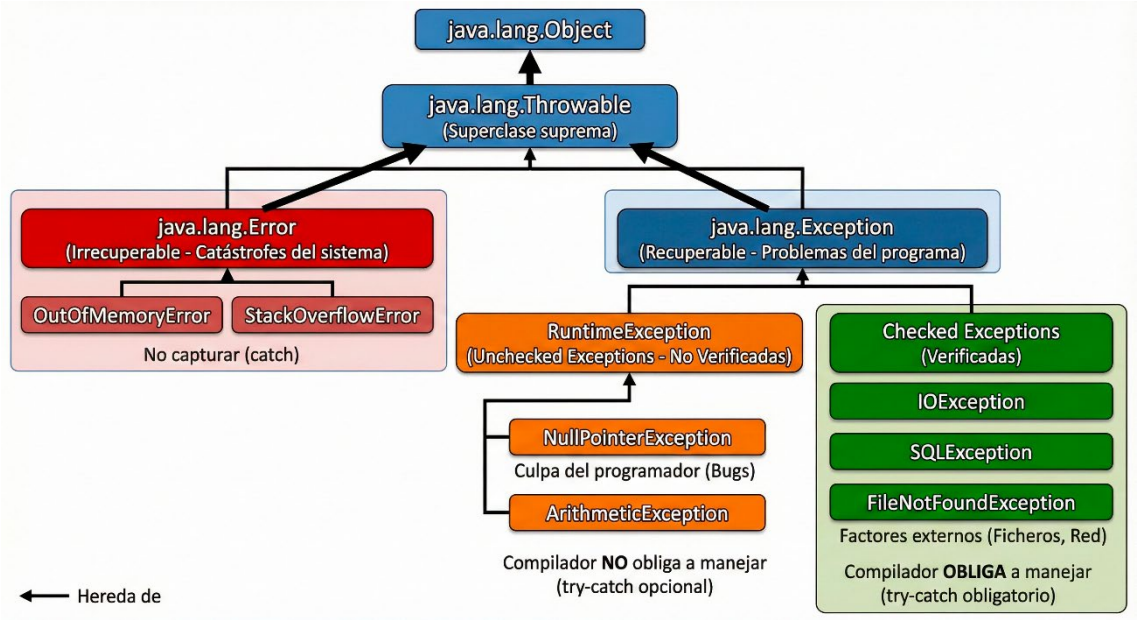
- **Comportamiento del Compilador:** El compilador **NO** te obliga a manejarlas. El código compila perfectamente, aunque no tengas un `try-catch`.
- **Causa:** Casi siempre son **culpas del programador** (bugs). Indican lógica incorrecta o uso indebido de una API.
- **Filosofía:** No deberías capturarlas para "ocultarlas", deberías **arreglar el código** para que no ocurran.
- **Ejemplos Clásicos:**

- `NullPointerException`: Intentar usar un objeto que es `null`.
- `ArrayIndexOutOfBoundsException`: Acceder a la posición 10 de un array de 5 elementos.
- `ArithmeticException`: Dividir por cero.

B. Checked Exceptions (Excepciones Verificadas)

Son todas las que heredan de `Exception` pero **NO** heredan de `RuntimeException`.

- **Comportamiento del Compilador:** El compilador **TE OBLIGA** a manejarlas. Si escribes código que puede lanzar una excepción de este tipo (como abrir un fichero) y no pones un `try-catch` (o declaras que la lanzas), **el código no compilará**. Te saldrá la famosa línea roja.
- **Causa:** Factores **externos** al programa que no puedes controlar totalmente (fallos de red, archivos, hardware externo).
- **Filosofía:** Java te dice: *"Sé que tu código está bien, pero el archivo podría no estar ahí. Te obligo a tener un plan B preparado"*.
- **Ejemplos Clásicos:**
  - `IOException`: Error de entrada/salida.
  - `SQLException`: Error en la base de datos.
  - `FileNotFoundException`: Archivo no encontrado.



Resumen Visual de Responsabilidad

Tipo	Clase Base	¿El compilador te obliga?	¿De quién es la culpa?	¿Cómo se soluciona?
Error	`Error`	No	Entorno / JVM	Aumentar recursos o

Tipo	Clase Base	¿El compilador te obliga?	¿De quién es la culpa?	¿Cómo se soluciona?
				rediseñar arquitectura.
Unchecked	`RuntimeException`	**No**	Programador (Tú)	**Corrigiendo el código** (bugs).
Checked	`Exception`	**Sí**	Entorno Externo	Usando **try-catch** para gestionar el fallo.

2. Manejo y Control de Excepciones

2.1. Captura de Excepciones: El bloque `try-catch-finally`

Esta es la estructura clásica para el manejo de errores en Java. Se compone de tres partes, cada una con una misión específica.

1. Estructura Clásica

- **`try` (Intentar):** Aquí encierras el **código peligroso**. Es el código que *podría* fallar (ej. abrir un archivo, conectar a una BD).
- **`catch` (Capturar):** Es el "plan de emergencia". Si algo falla dentro del `try`, el flujo salta inmediatamente al `catch`. Aquí escribes qué hacer para solucionar el problema o avisar al usuario.
- **`finally` (Finalmente):** Es el bloque de **limpieza**. Este código se ejecuta **SIEMPRE**, ocurra una excepción o no. Se usa para cerrar recursos (cerrar archivos, liberar memoria, cerrar conexiones).

Nota importante: El bloque `finally` es tan terco que se ejecuta incluso si dentro del `try` hay un `return`. (La única forma de evitar el `finally` es si se apaga la JVM brutalmente con `System.exit(0)`).

Ejemplo de sintaxis clásica:
Java
```
import java.util.Scanner;

public class EjemploClasico {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            // 1. CÓDIGO PELIGROSO
            System.out.println("Introduce un número para dividir
100:");
            int divisor = Integer.parseInt(scanner.nextLine());
```

```
            int resultado = 100 / divisor;
            System.out.println("El resultado es: " + resultado);

        } catch (ArithmeticException e) {
            // 2. MANEJO DEL ERROR (Solo si falla por matemáticas)
            System.out.println("¡Error! No puedes dividir por
cero.");

        } catch (NumberFormatException e) {
            // 2. MANEJO DEL ERROR (Solo si mete letras en vez de
números)
            System.out.println("¡Error! Eso no es un número
válido.");

        } finally {
            // 3. LIMPIEZA (Siempre ocurre)
            System.out.println("Cerrando el scanner y liberando
recursos...");
            scanner.close();
        }

        System.out.println("El programa continúa felizmente.");
    }
}
```

2. La evolución moderna: `try-with-resources` (Java 7+)

El bloque `finally` clásico tenía un problema: los programadores a menudo olvidaban cerrar los recursos (`.close()`), o peor aún, el cierre del recurso fallaba y generaba otra excepción dentro del `finally`, creando un caos de código ("Exception masking").

Para solucionar esto, Java 7 introdujo el **try-with-resources**.

¿Cómo funciona? En lugar de cerrar manualmente el recurso en el `finally`, declaras y abres el recurso **dentro de paréntesis justo después de la palabra** `try`.

- **Requisito:** El objeto debe implementar la interfaz `AutoCloseable` (casi todos los recursos de Java como archivos, sockets y bases de datos ya lo hacen).
- **Ventaja:** Java se encarga de cerrar el recurso automáticamente al terminar el bloque `try`, pase lo que pase. El código es mucho más limpio.

Comparativa: Leer la primera línea de un archivo

A) La forma antigua (Fea y verbosa):
Java
```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("archivo.txt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // Tienes que comprobar si es null antes de cerrar
    try {
        if (br != null) br.close();
```

```
    } catch (IOException ex) {
        ex.printStackTrace(); // ¡Un try-catch dentro del finally!
    }
}
```

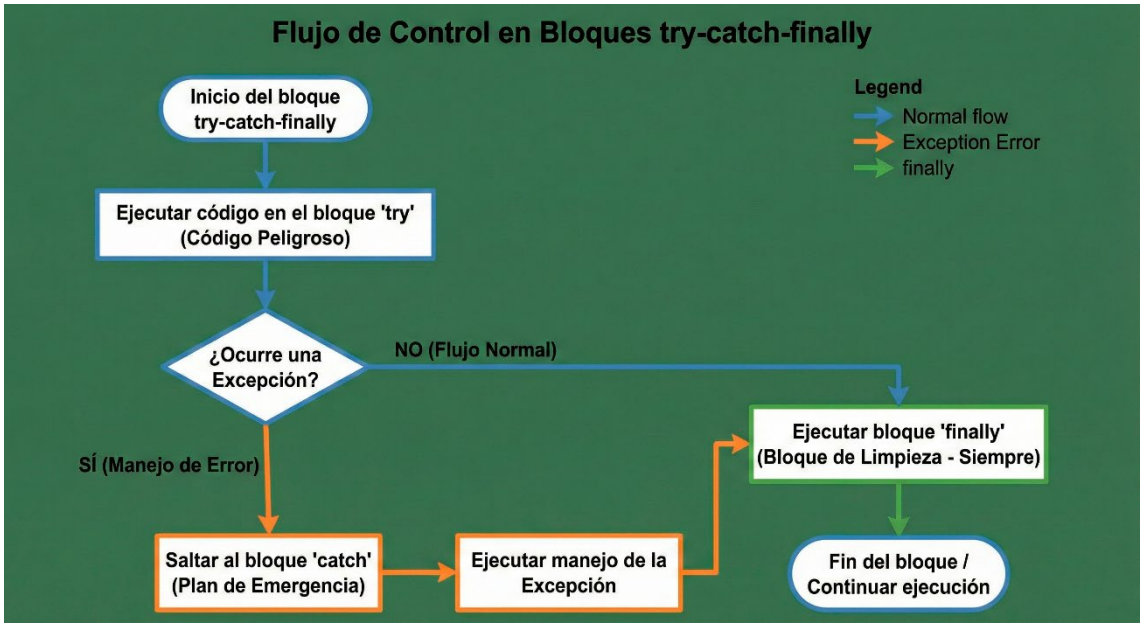
B) La forma moderna (`try-with-resources`):

Java

```
// Declaramos el recurso entre paréntesis del try
try (BufferedReader br = new BufferedReader(new
FileReader("archivo.txt"))) {

    System.out.println(br.readLine());

} catch (IOException e) {
    System.out.println("Hubo un error al leer el archivo.");
}
// ¡Magia! No hace falta finally. 'br' se cierra solo
automáticamente.
```



Diferencias Clave

Característica	Try-Catch-Finally (Clásico)	Try-with-resources (Moderno)
Cierre de recursos	Manual (en `finally`)	Automático (implícito)
Limpieza de código	Baja (verboso)	Alta (conciso)
Riesgo de fugas	Alto (si olvidas el `close`)	Nulo

Característica	Try-Catch-Finally (Clásico)	Try-with-resources (Moderno)
Uso principal	Lógica general	Cuando hay recursos externos (IO, BD)

2.2. Propagación: Uso de `throws` para delegar la responsabilidad

Hasta ahora hemos visto cómo capturar el error en el mismo lugar donde ocurre (`try-catch`). Pero, ¿qué pasa si el método donde ocurre el error **no sabe** o **no debe** decidir cómo solucionarlo?

Aquí entra en juego la **propagación de excepciones**.

El concepto de "Pasar la patata caliente"

Imagina una jerarquía en una empresa:

1. El **Gerente** le pide al **Jefe de Proyecto** un informe.
2. El **Jefe de Proyecto** le pide al **Programador** que escriba el código.
3. El **Programador** intenta escribir, pero **se le rompe el teclado**.

El programador no puede comprar teclados (no tiene presupuesto). Su responsabilidad no es solucionar el fallo de hardware, sino **avisar** (lanzar la excepción) a su jefe. El jefe recibe el problema; si él tampoco puede solucionarlo, se lo pasa al Gerente.

En Java, esto se hace declarando que un método "avisa" de que puede fallar, usando la palabra clave **`throws`** en su firma.

Sintaxis: La cláusula `throws`

Cuando defines un método, puedes añadir `throws` seguido del tipo de excepción al final de la cabecera. Esto es una **advertencia obligatoria** para quien llame a ese método.

Java
```
// Este método NO captura el error, avisa de que puede ocurrir.
public void leerArchivoImportante(String ruta) throws IOException {
    // Código que lee el archivo y puede fallar
    FileReader file = new FileReader(ruta);
    // ...
}
```

Mecanismo de Propagación (Call Stack)

Cuando ocurre una excepción:

1. Java busca un `catch` en el método actual.
2. Si no lo encuentra (porque usamos `throws`), el método termina abruptamente y la excepción **burbujea hacia arriba** al método que lo llamó (el "caller").

3. Este proceso se repite hasta que alguien la captura o hasta que llega al `main()`. Si el `main()` también la propaga (con `throws`), el programa termina y la JVM imprime la traza del error en la consola.

Ejemplo Práctico: Delegar vs. Manejar

Veamos la diferencia entre solucionar el problema tú mismo o pasárselo a otro.

Escenario: Queremos leer un archivo.

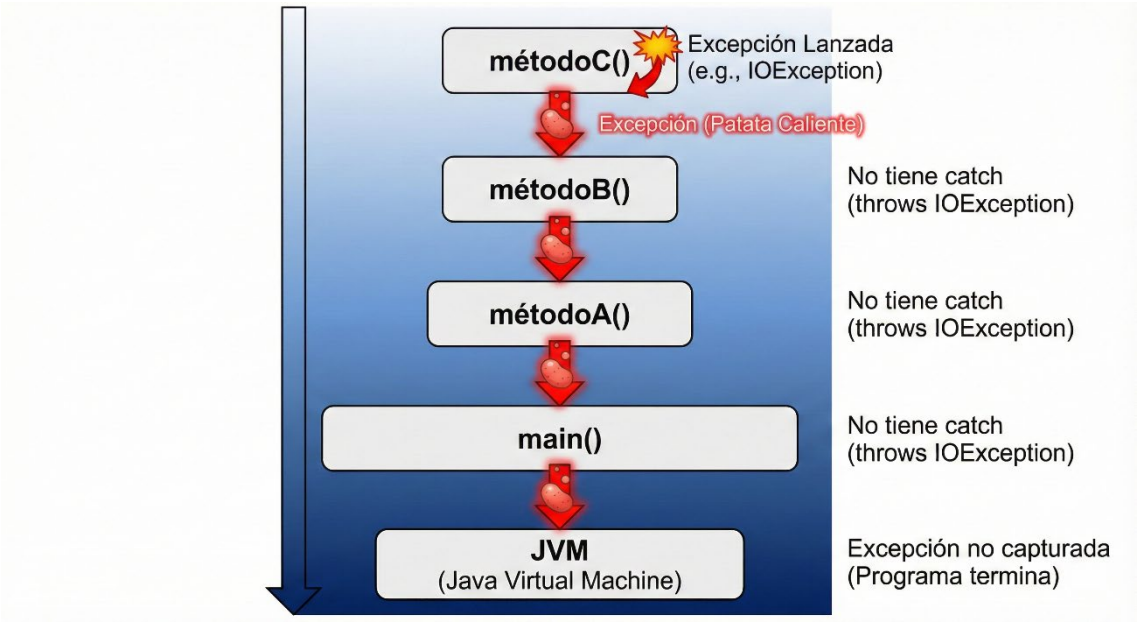
Opción A: El método maneja el error (El "solucionador"). El método se hace cargo. Quien llame a este método ni se entera de que hubo un riesgo.

Java
```
public void cargarConfiguracion() {
    try {
        FileReader fr = new FileReader("config.txt");
    } catch (FileNotFoundException e) {
        System.out.println("Creando archivo por defecto..."); //
Solución local
    }
}
```

Opción B: El método propaga el error (El "delegador"). El método dice: "Yo solo sé leer. Si el archivo no está, que decida qué hacer quien me llamó".

Java
```
// Nótese el 'throws' aquí
public void leerDatos() throws FileNotFoundException {
    FileReader fr = new FileReader("datos.txt"); // Si falla,
explota hacia arriba
}

// El método que llama es quien está obligado a gestionar la crisis
public void main(String[] args) {
    try {
        leerDatos();
    } catch (FileNotFoundException e) {
        System.out.println("El usuario olvidó el archivo.
Pidiéndoselo de nuevo...");
    }
}
```



Diferencia entre `throw` y `throws`

Es vital no confundir estas dos palabras, aunque se parecen mucho:

Palabra Clave	Significado	Ubicación	Acción
`throws`	**Advertencia**: "Peligro, este método *podría* lanzar un error".	En la **firma** del método (cabecera).	**Declara** una posibilidad. Delega la responsabilidad.
`throw`	**Acción**: "¡Lanza el error ahora mismo!".	Dentro del **cuerpo** del método (código).	**Provoca** la excepción explícitamente.

Regla de oro para Checked Exceptions: Si tienes código que lanza una excepción verificada (como `IOException`), tienes solo dos opciones: o la capturas con `try-catch` **O** la declaras en la firma con `throws`. No puedes ignorarla.

2.3. Lanzamiento de Excepciones: Uso de `throw`

Hasta ahora, las excepciones las generaba "automáticamente" la Máquina Virtual de Java (JVM) cuando detectaba algo imposible (como dividir por cero o acceder a un objeto nulo).

Sin embargo, en el mundo real, **tú** eres quien define qué es un error en tu programa. Aquí entra la palabra clave **`throw`**.

¿Qué hace `throw`?

La instrucción `throw` se utiliza para **disparar explícitamente** una excepción. Es la forma que tienes de decir: *"¡Alto! Aquí ha ocurrido algo que viola las reglas de mi programa, detengan todo"*.

Piensa en un árbitro de fútbol. Si un jugador comete una falta, el juego no se detiene por leyes físicas (el jugador sigue corriendo), se detiene porque el árbitro **sopla el silbato** (`throw`) para indicar una infracción.

Sintaxis

Para lanzar una excepción, necesitas crear una instancia (un objeto) de la clase de excepción que quieras usar (usando `new`) y "lanzársela" a la JVM.
Java
```
// Sintaxis: throw + new + TipoDeExcepcion("Mensaje explicativo");
throw new IllegalArgumentException("La edad no puede ser
negativa");
```

¿Por qué querrías provocar un error? (Casos de uso)

La razón principal es la **Validación de Reglas de Negocio**. Java no sabe que una edad negativa es imposible, o que no puedes retirar más dinero del que tienes en el banco. Para Java, `-5` es un número entero válido. Tú usas `throw` para imponer tus reglas.

Ejemplo Práctico: Setter con validación

Imagina una clase `Persona`. Queremos evitar que alguien asigne una edad absurda.

Java
```
public class Persona {
    private int edad;

    public void setEdad(int edad) {
        // 1. Validamos ANTES de asignar
        if (edad < 0) {
            // Violación de regla: Lanzamos la excepción
manualmente
            throw new IllegalArgumentException("¡Error! La edad ("
+ edad + ") no puede ser negativa.");
        }

        // 2. Si pasa el filtro, asignamos
        this.edad = edad;
    }
}
```

¿Qué ocurre al ejecutar esto?
Java
```
Persona p = new Persona();
p.setEdad(-10); // ¡PUM! El programa se detiene y muestra tu
mensaje.
// Exception in thread "main" java.lang.IllegalArgumentException:
¡Error! La edad (-10) no puede ser negativa.
```

`throw` vs. `throws`: La distinción final

Es el error más común en los exámenes y entrevistas. Vamos a dejarlo cristalino:

Palabra Clave	Función	¿Dónde se pone?	Analogía
`throw`	**Acción**. Ejecuta el lanzamiento del error.	Dentro del código (instrucción).	El árbitro pitando la falta.
`throws`	**Aviso**. Declara que un método *podría* fallar.	En la firma del método.	El cartel de "Cuidado con el perro" en la puerta.

Buenas Prácticas al usar `throw`

1. **Usa excepciones estándar cuando puedas:** Java ya tiene excepciones para casi todo.

   - Si el argumento es nulo: `throw new NullPointerException();`
   - Si el argumento no tiene sentido (ej. edad negativa): `throw new IllegalArgumentException();`
   - Si el estado del objeto no permite la acción: `throw new IllegalStateException();`

2. **Mensajes claros:** Escribe siempre un mensaje descriptivo dentro del constructor de la excepción. Ayudará mucho a quien tenga que depurar el error ("La lista está vacía" es mejor que "Error").

2.4. Creación de Excepciones Personalizadas

Crear una excepción personalizada (Custom Exception) te permite **adaptar los errores al lenguaje de tu dominio**. En lugar de lanzar un genérico `IllegalArgumentException`, puedes lanzar un `SaldoInsuficienteException`. Esto hace que tu código sea mucho más legible y fácil de mantener.

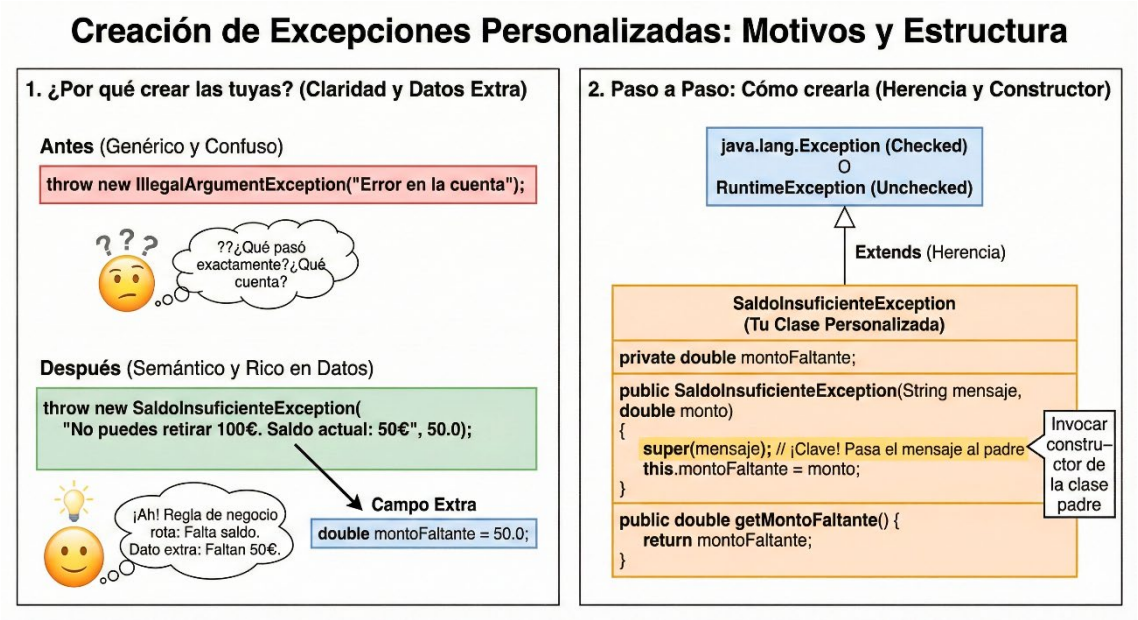
¿Por qué crear las tuyas?

1. **Claridad Semántica:** El nombre del error te dice exactamente qué regla de negocio se rompió.
2. **Información Extra:** Puedes añadir campos a tu excepción (por ejemplo, `montoFaltante` en un error de saldo) para ayudar a solucionar el problema.

Paso a Paso: Cómo crearla

Para crear tu propia excepción, simplemente creas una clase nueva y aplicas **Herencia**. Tienes dos opciones principales:

1. **Extender de `Exception`:** Creas una **Checked Exception**. Obligas a quien use tu método a poner un `try-catch`. Es ideal para reglas de negocio importantes donde quieres forzar al programador a manejar el error (ej: "Saldo insuficiente").
2. **Extender de `RuntimeException`:** Creas una **Unchecked Exception**. No obligas a capturarla. Se usa más para errores técnicos o estados irrecuperables.



La Estructura Básica

Lo más importante es invocar al constructor de la clase padre (`super`) para pasarle el mensaje de error.

Java
```
// 1. Definimos la clase (El nombre debe terminar en 'Exception')
public class SaldoInsuficienteException extends Exception {

    // 2. Constructor que recibe el mensaje explicativo
    public SaldoInsuficienteException(String mensaje) {
        // 3. Pasamos el mensaje a la clase padre (Exception)
        super(mensaje);
    }

    // Opcional: Constructor vacío
    public SaldoInsuficienteException() {
        super("Saldo insuficiente para realizar la operación.");
    }
}
```

Ejemplo Completo: El Cajero Automático

Vamos a ver cómo se integran todas las piezas: la Excepción Personalizada, el lanzamiento (`throw`) y la declaración (`throws`).

1. La Clase de Negocio (Cuenta Bancaria)

Java
```
public class CuentaBancaria {
```

```
    private double saldo;

    public CuentaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    // Aviso: Este método PUEDE fallar y lanzar nuestra excepción
    public void retirar(double cantidad) throws
SaldoInsuficienteException {
        if (cantidad > saldo) {
            double faltante = cantidad - saldo;
            // Detenemos todo lanzando nuestra excepción
personalizada
            throw new SaldoInsuficienteException("Te faltan " +
faltante + " euros para retirar esa cantidad.");
        }

        saldo -= cantidad;
        System.out.println("Retiro exitoso. Nuevo saldo: " +
saldo);
    }
}
```

2. La Clase Principal (Main)

Java

```
public class CajeroAutomatico {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria(100.0); //
Tienes 100€

        try {
            // Intentamos sacar 500€ (Ups...)
            cuenta.retirar(500.0);

        } catch (SaldoInsuficienteException e) {
            // Capturamos NUESTRA excepción específica
            System.out.println("Error en la transacción:");
            System.out.println(e.getMessage()); // Muestra el
mensaje que escribimos arriba

        } catch (Exception e) {
            // Captura genérica por si pasa algo más raro
            System.out.println("Ocurrió un error inesperado.");
        }
    }
}
```

Salida en consola:

Plaintext

```
Error en la transacción:
Te faltan 400.0 euros para retirar esa cantidad.
```

Ventajas de este enfoque

Si hubiéramos usado una excepción genérica como `RuntimeException`, el programador que usa la clase `CuentaBancaria` podría no saber qué hacer si falla. Al usar

`SaldoInsuficienteException`, el código se "autodocumenta": *"Si llamas a retirar, prepárate porque puede que no haya saldo"*.

Buenas Prácticas

- **Naming Convention:** Termina siempre el nombre de tu clase con la palabra `Exception` (ej: `InvalidUserException`).
- **Jerarquía:** Si es un evento del que el usuario se puede recuperar (como poner mal una contraseña o falta de saldo), extiende de `Exception`. Si es un fallo de programación (como índice negativo), extiende de `RuntimeException`.
- **Constructores:** Implementa al menos dos constructores: uno vacío y otro que acepte un `String message`.

3. El Java Collections Framework (JCF)

3.1. Tipos de Colecciones y Estructuras

El JCF se basa en interfaces. No te preocupas tanto por *cómo* se guardan los datos, sino por *cómo se comportan*. La jerarquía principal se divide en cuatro grandes familias.

1. Listas (`List`): El orden importa

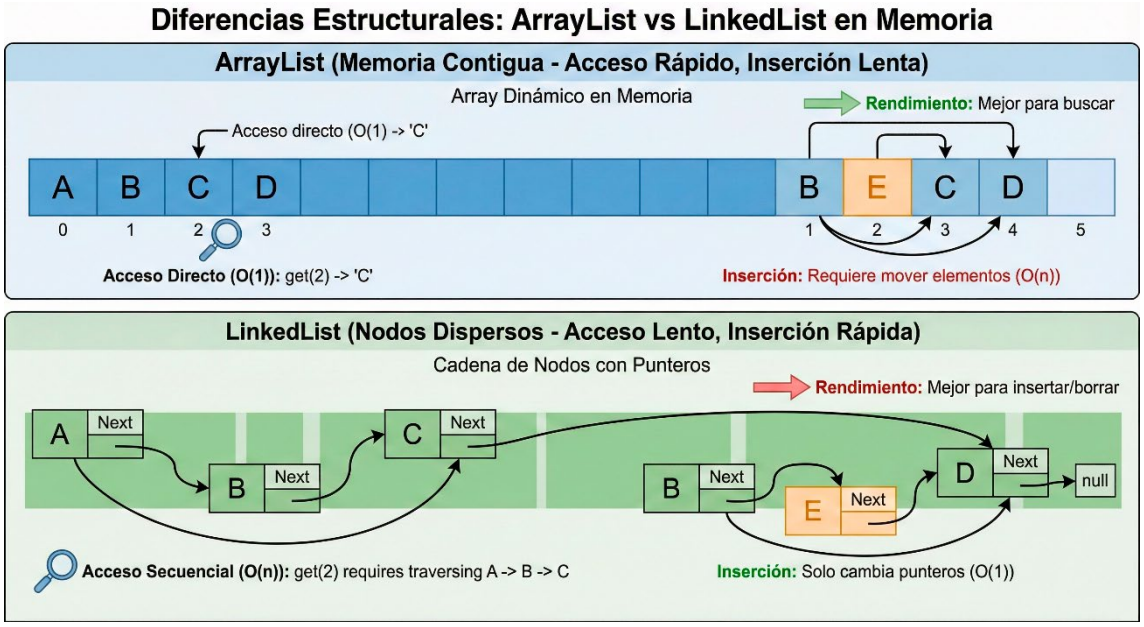
Una lista es una **secuencia ordenada** de elementos.

- **Características:**
  - **Orden:** Los elementos mantienen el orden en que los insertas.
  - **Acceso:** Puedes acceder a cualquier elemento por su índice (posición `0`, `1`, etc.).
  - **Duplicados: Permite** elementos repetidos (puedes tener dos veces el número `5`).

Las dos implementaciones clave:

- **`ArrayList`:** Es básicamente un array dinámico que crece solo.
  - **Mejor para:** Buscar y leer datos (`get(5)` es rapidísimo).
  - **Peor para:** Insertar o borrar elementos en medio de la lista (tiene que mover todos los demás elementos).
- **`LinkedList`:** Es una cadena de nodos donde cada uno apunta al siguiente.
  - **Mejor para:** Insertar y borrar datos frecuentemente (solo cambias los punteros).
  - **Peor para:** Buscar datos (tiene que recorrer la lista desde el principio).

Regla práctica: Usa `ArrayList` por defecto (90% de los casos). Usa `LinkedList` solo si vas a estar borrando e insertando datos constantemente.



2. Conjuntos (`Set`): La unicidad manda

Un conjunto es una colección que **no permite duplicados**. Es como una bolsa de canicas; no importa en qué orden las metiste, lo que importa es si la canica está ahí o no.

- **Características:**
  - **Unicidad:** Si intentas añadir un elemento que ya existe, simplemente lo ignora (devuelve `false`).
  - **Uso:** Ideal para eliminar repetidos o verificar pertenencia (ej: correos electrónicos de usuarios, IDs).

Las dos implementaciones clave:

- `HashSet`:
  - **Orden: Caótico**. No garantiza ningún orden específico.
  - **Rendimiento:** Extremadamente rápido (gracias a una tabla hash interna).
- `TreeSet`:
  - **Orden: Ordenado**. Mantiene los elementos ordenados naturalmente (alfabéticamente o numéricamente).
  - **Rendimiento:** Más lento que HashSet.

3. Colas (`Queue`) y Pilas (`Stack`)

Aquí lo importante no es el índice, sino el **orden de procesamiento**.

Colas (Queue) - FIFO (First In, First Out)

Como la fila del supermercado. El primero que llega es el primero en ser atendido.

- **Implementación:** Se suele usar `LinkedList` o `PriorityQueue` (ordenada por prioridad).

- **Métodos clave:** `offer()` (entrar), `poll()` (salir y borrar), `peek()` (mirar quién es el siguiente).

Pilas (Stack) - LIFO (Last In, First Out)

Como una pila de platos para lavar. El último plato que pones encima es el primero que quitas.

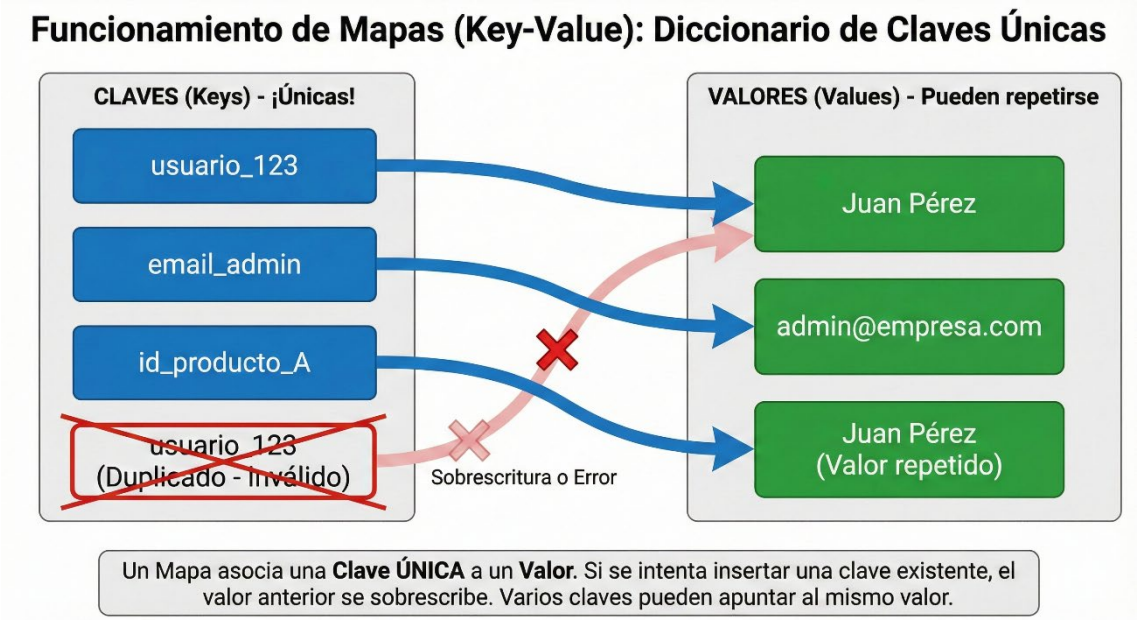
- **Nota moderna:** La clase `Stack` es antigua en Java. Hoy en día se recomienda usar la interfaz **`Deque`** (Double Ended Queue) con la implementación **`ArrayDeque`**.

4. Mapas (`Map`): Diccionarios Clave-Valor

¡Ojo! Los Mapas **NO** heredan de la interfaz `Collection`. Son una rama separada en el diagrama, pero son parte vital del framework.

Un mapa asocia **Claves (Keys)** con **Valores (Values)**. Es como un diccionario: buscas una palabra (clave) para obtener su definición (valor).

- **Características:**
  - **Claves únicas:** No puede haber dos claves iguales. Si repites una clave, sobrescribes el valor anterior.
  - **Valores repetidos:** Varios claves pueden apuntar al mismo valor.



Las dos implementaciones clave:

- **`HashMap`:**
  - El estándar. Rápido, eficiente, sin orden garantizado de las claves.
- **`TreeMap`:**
  - Mantiene las **claves ordenadas**. Útil si necesitas listar el diccionario en orden alfabético.

Resumen: ¿Qué elijo?

¿Qué necesitas?	Interfaz	Implementación recomendada
Una lista simple, acceso rápido	`List`	**`ArrayList`**
Lista con muchas inserciones/borrados	`List`	`LinkedList`
Sin duplicados, lo más rápido posible	`Set`	**`HashSet`**
Sin duplicados, ordenado (A-Z, 1-10)	`Set`	`TreeSet`
Diccionario Clave-Valor rápido	`Map`	**`HashMap`**
Diccionario Ordenado por clave	`Map`	`TreeMap`
Procesar en orden de llegada (FIFO)	`Queue`	`LinkedList` / `ArrayDeque`
Procesar lo último primero (LIFO)	`Deque`	**`ArrayDeque`**

3.2. Jerarquía de la Interfaz Collection

El Java Collections Framework (JCF) está organizado como un árbol genealógico de **Interfaces**. Esto significa que define *contratos* de comportamiento antes que implementaciones concretas.

Aquí tienes el mapa visual de las relaciones que vamos a desglosar:

1. La Raíz: `Iterable<T>`

Todo comienza aquí. Es la "bisabuela" de todas las colecciones (excepto los Mapas).

- **Función:** Cualquier clase que implemente `Iterable` **puede ser recorrida** usando un bucle **"for-each"**.
- **Método clave:** `iterator()`.
- **Consecuencia:** Si tu objeto es `Iterable`, puedes hacer esto:

Java
```
for (String elemento : miColeccion) { ... }
```

2. El Tronco Principal: `Collection<T>`

Es la interfaz principal que agrupa a Listas, Conjuntos y Colas. Hereda directamente de `Iterable`.

- **Función:** Define los comportamientos universales para cualquier grupo de objetos, sin importar si están ordenados o repetidos.
- **Métodos Universales:** Si usas `Collection` como tipo de dato, tienes garantizados métodos como:
  - `add(E e)` (Añadir)
  - `remove(Object o)` (Borrar)
  - `size()` (Tamaño)

- `contains(Object o)` (¿Existe?)
- `isEmpty()` (¿Está vacía?)
- `clear()` (Borrar todo)

Poder del Polimorfismo: Si escribes un método que recibe un `Collection<String>`, ese método aceptará tanto un `ArrayList` como un `HashSet`. ¡Código reutilizable!

3. Las Ramas (Hijas de Collection)

Aquí es donde la funcionalidad se especializa. Todas heredan de `Collection`, por lo que tienen todos los métodos anteriores, más los suyos propios.

A. `List<T>` (Hija Ordenada)

- **Hereda de:** `Collection`.
- **Lo nuevo:** Introduce el concepto de **índice** (posiciones numéricas).
- **Métodos exclusivos:** `get(int index)`, `set(int index, E element)`, `indexOf(Object o)`.

B. `Set<T>` (Hija Estricta)

- **Hereda de:** `Collection`.
- **Lo nuevo:** No añade métodos nuevos significativos sobre `Collection`, pero **cambia el contrato** del método `add()`. Si intentas añadir un duplicado, `add()` devuelve `false` y no lo inserta.

4. El "Vecino": `Map<K, V>` (La Gran Excepción)

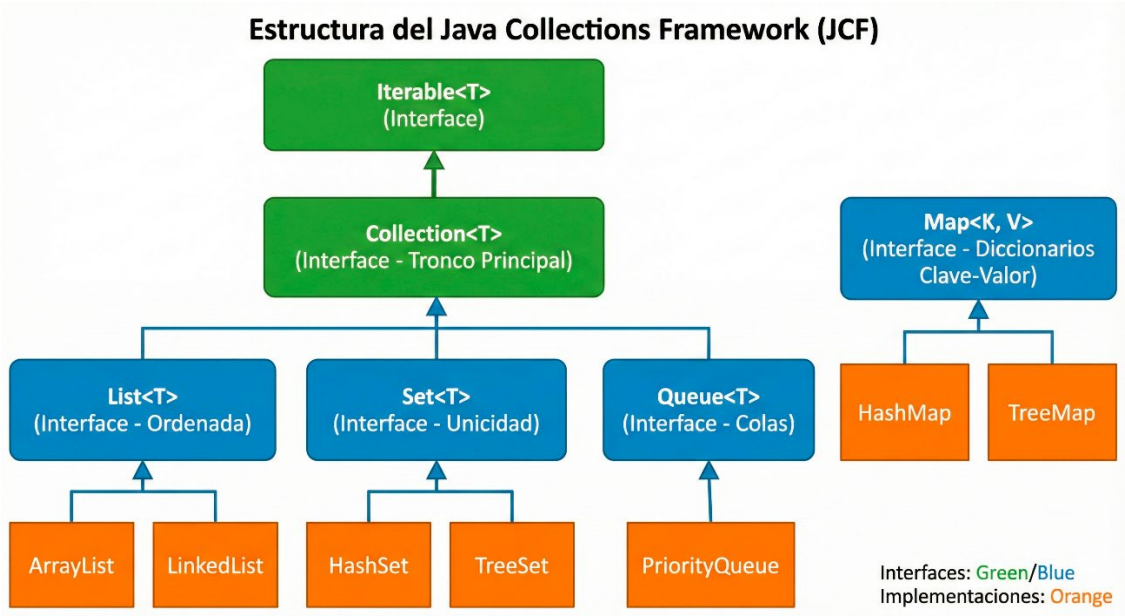
Esta es la distinción más importante de la arquitectura y una pregunta clásica de entrevista.

> **Importante:** La interfaz `Map` **NO hereda de `Collection` ni de `Iterable`.**

¿Por qué? Porque `Collection` trabaja con elementos individuales (objetos "sueltos"), mientras que `Map` trabaja con **parejas** (Clave-Valor). Sus estructuras son incompatibles matemáticamente.

- **¿Cómo se relaciona entonces?** Aunque son independientes, los Mapas proporcionan "Vistas" (`Views`) que sí son Colecciones para poder interactuar con ellos:

  1. `keySet()`: Devuelve un `Set` con todas las claves.
  2. `values()`: Devuelve una `Collection` con todos los valores.
  3. `entrySet()`: Devuelve un `Set` de pares clave-valor.



Resumen de Relaciones

Interfaz	¿Hereda de Iterable?	¿Hereda de Collection?	Concepto Clave
Iterable	-	No	Permite `for-each`.
Collection	**Sí**	-	Grupo genérico de cosas.
List	**Sí**	**Sí**	Cosas ordenadas por índice.
Set	**Sí**	**Sí**	Cosas únicas (sin repetidos).
Map	**No**	**No**	Diccionario (Parejas Clave-Valor).

Ejemplo de código: La Jerarquía en acción

Java
```
import java.util.*;

public class JerarquiaDemo {
    public static void main(String[] args) {
        // Polimorfismo: Usamos la interfaz padre 'List' para la
referencia
        List<String> lista = new ArrayList<>();
        lista.add("Java");
        lista.add("Python");

        // Polimorfismo: Usamos la interfaz padre 'Set'
        Set<String> conjunto = new HashSet<>();
```

```
        conjunto.add("Java");

        // Un método que acepta 'Collection' funciona para ambos
        imprimirTamaño(lista);       // Funciona
        imprimirTamaño(conjunto);    // Funciona

        // Map va por libre
        Map<Integer, String> mapa = new HashMap<>();
        mapa.put(1, "Java");
        // imprimirTamaño(mapa); // ¡ERROR DE COMPILACIÓN! Map no
es Collection
    }

    // Este método es muy flexible gracias a la jerarquía
    public static void imprimirTamaño(Collection<?> c) {
        System.out.println("El tamaño es: " + c.size());
        // Como es Collection, también es Iterable:
        for(Object o : c) {
            System.out.println("- " + o);
        }
    }
}
```

4. Operaciones y Tipado

4.1. Operaciones con Colecciones

Vamos a dividir las operaciones básicas (CRUD: Create, Read, Update, Delete) según el tipo de colección, ya que la sintaxis cambia.

A. Inserción, Acceso y Eliminación

Operación	**Listas (`List`)**	**Conjuntos (`Set`)**	**Mapas (`Map`)**
Insertar	`add(elemento)` `add(índice, elemento)`	`add(elemento)` *(Devuelve `false` si ya existe)*	`put(clave, valor)` *(Si la clave existe, sobrescribe)*
Acceder	`get(índice)`	No tiene acceso directo. Se usa `contains(elemento)`	`get(clave)` *(Devuelve `null` si no existe)*
Eliminar	`remove(índice)`	`remove(objeto)`	`remove(clave)`

Operación	Listas (`List`)	Conjuntos (`Set`)	Mapas (`Map`)
	`remove(objeto)`		
Tamaño	`size()`	`size()`	`size()`

Ejemplo comparativo rápido:
Java
```
// LISTA
List<String> lista = new ArrayList<>();
lista.add("Juan");           // Inserción
String nombre = lista.get(0); // Acceso por posición
lista.remove(0);             // Eliminación por posición

// MAPA (Nota el cambio de sintaxis)
Map<Integer, String> mapa = new HashMap<>();
mapa.put(1, "Juan");         // Inserción (Clave, Valor)
String valor = mapa.get(1); // Acceso por Clave
mapa.remove(1);              // Eliminación por Clave
```

B. Recorridos: Iterando sobre los datos

Una vez tenemos una colección llena de datos, a menudo necesitamos pasar por cada uno de ellos para procesarlos (sumarlos, imprimirlos, filtrarlos). Java ha evolucionado ofreciendo tres formas de hacerlo.

1. El Clásico: `Iterator` (La forma segura de borrar)

Es el mecanismo subyacente de Java. Un iterador es un objeto "puntero" que se mueve sobre la colección.

Ventaja clave: Es la **única forma segura de eliminar elementos mientras recorres la lista**.

Sintaxis:
Java
```
List<String> nombres = new ArrayList<>(Arrays.asList("Ana",
"Pedro", "Luis"));

Iterator<String> it = nombres.iterator();
while(it.hasNext()) {
    String nombre = it.next();
    if (nombre.equals("Pedro")) {
        it.remove(); // ¡SEGURO! Elimina el elemento actual
correctamente
    }
}
```

2. El Estándar: Bucle `for-each` (Syntactic Sugar)

Introducido en Java 5. Es la forma más limpia y legible de leer datos. Internamente usa un `Iterator`, pero nos oculta la complejidad.

Limitación: Es de **solo lectura**. Si intentas borrar un elemento de la lista (`lista.remove(n)`) dentro de un `for-each`, obtendrás una excepción `ConcurrentModificationException`.

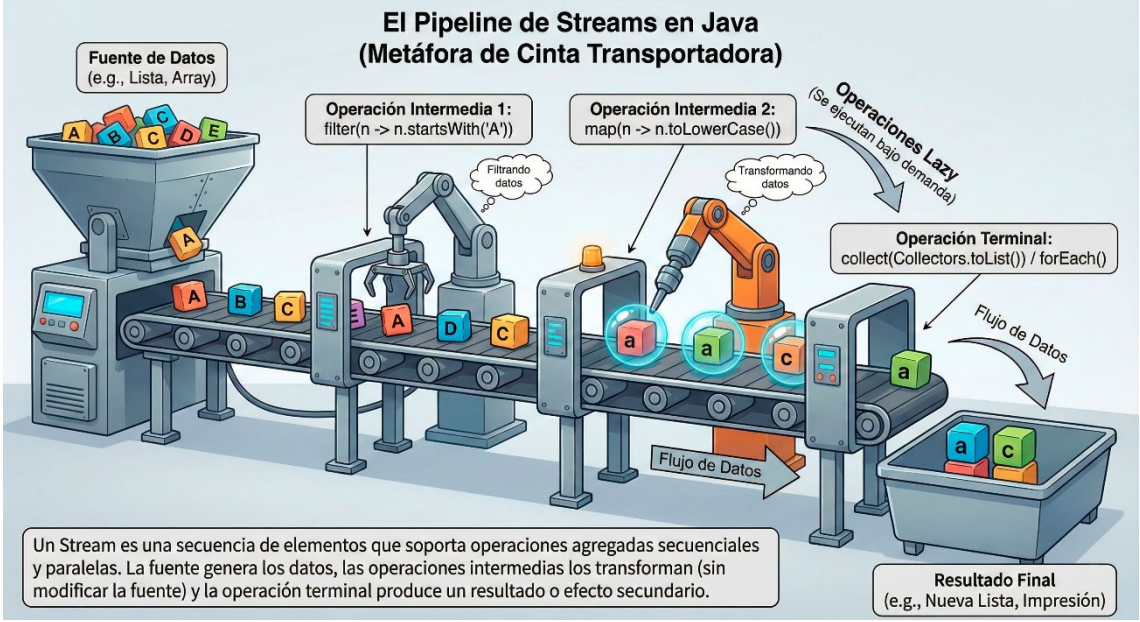
Sintaxis:
Java
```
for (String nombre : nombres) {
    System.out.println("Procesando a: " + nombre);
    // nombres.remove(nombre); // ¡ERROR! No hagas esto aquí.
}
```

3. La Era Moderna: Introducción a los `Streams` (Java 8+)

Los **Streams** (Flujos) cambiaron la forma de programar en Java hacia un estilo **funcional** y declarativo. En lugar de decir *cómo* recorrer la lista (con bucles), dices *qué* quieres hacer con los datos.

Un Stream es como una cinta transportadora de datos donde puedes encadenar operaciones.

Concepto: Fuente de datos -> Operaciones Intermedias (filtro, transformación) -> Operación Terminal (resultado).



Sintaxis (Introducción):
Java
```
List<String> nombres = Arrays.asList("Ana", "Alberto", "Bea",
"Antonio");

// Queremos imprimir solo los nombres que empiezan por 'A'
nombres.stream()
```

```
    .filter(n -> n.startsWith("A"))  // 1. Filtramos (Operación
Intermedia)
    .map(n -> n.toUpperCase())       // 2. Transformamos a
mayúsculas (Op. Intermedia)
    .forEach(n -> System.out.println(n)); // 3. Acción final (Op.
Terminal)

// Salida: ANA, ALBERTO, ANTONIO
```

Más info: Los mapas no son iterables directamente. Para recorrerlos con un `for-each`, debes iterar sobre su conjunto de entradas (`entrySet()`):
Java

```
for (Map.Entry<Integer, String> entrada : mapa.entrySet()) {
    System.out.println("Clave: " + entrada.getKey() + " Valor: " +
entrada.getValue());
}
```

Resumen de Recorridos

Método	Legibilidad	Capacidad de Modificar	Estilo
Iterator	Baja (Verboso)	**Alta** (Permite borrar)	Imperativo (Paso a paso)
For-each	**Muy Alta**	Nula (Solo lectura)	Imperativo simplificado
Stream	Alta (Declarativo)	Nula (Crea nuevos resultados)	**Funcional** (Pipeline)

4.2. Uso de Clases y Métodos Genéricos en Colecciones

En la unidad anterior aprendiste que `<T>` es un "marcador de posición" para un tipo de dato. En el *Java Collections Framework*, aplicamos esto para definir **qué tipo exacto de objetos** permitimos dentro de nuestra estructura.

1. El problema de la "Antigua Escuela" (Raw Types)

Antes de Java 5, las colecciones no tenían tipos. Todo era una lista de `Object`. Esto significaba que una `ArrayList` era como una caja de mudanza sin etiquetar: podías meter libros, platos y al gato en la misma caja.

El problema surgía al **sacar** las cosas:

1. Tenías que recordar qué había dentro.
2. Tenías que hacer un **Casting manual** (convertir `Object` a lo que creías que era).
3. Si te equivocabas, el programa explotaba en tiempo de ejecución (`ClassCastException`).

2. La Solución: Colecciones Seguras ("Type Safety")

Al usar Genéricos (`List<String>`), le dices al compilador: *"Esta lista SOLO acepta Cadenas de texto"*. El compilador actúa como un guardia de seguridad.

Comparativa: Sin Genéricos vs. Con Genéricos

A) Sin Genéricos (Peligroso y Verboso)

Java
```
// Lista "cruda" (Raw type) - Acepta Object
List lista = new ArrayList();

lista.add("Hola");
lista.add(123); // ¡Deja meter un entero! Peligro.

// Al sacar, necesitamos Casting explícito
String texto = (String) lista.get(0);

// ¡ERROR EN TIEMPO DE EJECUCIÓN!
// Java intenta convertir el 123 a String y falla. El programa se
cierra.
String numero = (String) lista.get(1); // ClassCastException
```

B) Con Genéricos (Seguro y Limpio)

Java
```
// Definimos que T = String
List<String> lista = new ArrayList<>();

lista.add("Hola");
// lista.add(123); // ¡ERROR DE COMPILACIÓN! El IDE te marca rojo.
No compila.

// Al sacar, NO hace falta casting. Java ya sabe que es un String.
String texto = lista.get(0);
```

Beneficio clave: Transformamos los errores de ejecución (que ocurren cuando el cliente usa la app) en errores de compilación (que ocurren mientras tú programas).

3. El Operador Diamante (<>)

Desde Java 7, la sintaxis se simplificó. Como ya declaraste el tipo a la izquierda (`List<String>`), no hace falta repetirlo a la derecha. Basta con poner los paréntesis angulares vacíos `<>`.

Java
```
// Java 5/6 (Redundante)
Map<String, Usuario> usuarios = new HashMap<String, Usuario>();

// Java 7+ (Operador Diamante - Inferencia de tipos)
Map<String, Usuario> usuarios = new HashMap<>();
```

4. Métodos Genéricos en Colecciones

No solo las clases son genéricas, también los métodos. Las colecciones suelen trabajar con métodos que aceptan `<T>` para ser flexibles.

Imagina que quieres un método que imprima cualquier lista, sea de `String`, `Integer` o `Usuario`.

Java

```
// El <T> antes del void indica que vamos a usar un tipo genérico
en este método
public static <T> void imprimirLista(List<T> lista) {
    for (T elemento : lista) {
        System.out.println(elemento);
    }
}

public static void main(String[] args) {
    List<String> nombres = Arrays.asList("Ana", "Juan");
    List<Integer> numeros = Arrays.asList(1, 2, 3);

    imprimirLista(nombres); // T se convierte en String
automáticamente
    imprimirLista(numeros); // T se convierte en Integer
automáticamente
}
```

Ventajas

1. **Seguridad de Tipos (Type Safety):** El compilador garantiza que no metes peras en la cesta de manzanas.
2. **Eliminación del Casting:** El código es más limpio porque no necesitas escribir `(String)` o `(Persona)` cada vez que lees datos.
3. **Legibilidad:** Al ver `List<Factura>`, cualquier programador sabe exactamente qué contiene esa variable sin tener que leer todo el código.

Resumen

Manejo de Excepciones (Haciendo el código robusto)

El objetivo no es evitar que ocurran errores (eso es imposible), sino gestionarlos para que el programa no colapse.

1. La Jerarquía de Errores

Todo desciende de **`Throwable`**.

- **`Error`:** Problemas graves de la JVM (memoria agotada). **Irrecuperables**.
- **`Exception`:** Problemas del programa. **Recuperables**.
    - **Checked (`Exception`):** Factores externos (ficheros, red). El compilador **te obliga** a manejarlas.
    - **Unchecked (`RuntimeException`):** Errores de programación (bugs, nulos, dividir por cero). El compilador **no** te obliga.

2. Las 5 Palabras Clave

Palabra	Función	Analogía
`try`	Envuelve el código peligroso.	Zona de pruebas.
`catch`	Define qué hacer si ocurre un error específico.	Plan de emergencia.
`finally`	Código que se ejecuta SIEMPRE (limpieza).	El equipo de limpieza.
`throw`	Lanza una excepción manualmente (`new`).	Pitar una falta.
`throws`	Avisa en la firma del método que puede fallar.	Cartel de "Peligro".

3. Buenas Prácticas

- Usar `try-with-resources` para cerrar automáticamente flujos de datos (archivos, conexiones) y evitar fugas de memoria.
- Crear **Excepciones Personalizadas** para modelar errores de tu lógica de negocio (ej. `SaldoInsuficienteException`).

Java Collections Framework (Gestión eficiente de datos)

Dejamos atrás los arrays estáticos y rígidos para usar estructuras dinámicas y flexibles.

1. Las Interfaces Principales

Interfaz	Característica Clave	Implementación Típica	Uso Ideal
`List`	Ordenada, permite duplicados.	`ArrayList`	Listas de la compra, historiales.
`Set`	Sin duplicados, unicidad.	`HashSet`	Ids únicos, correos, inventario.
`Map`	Diccionario Clave-Valor.	`HashMap`	Agendas, cachés, configuraciones.
`Queue`	Orden de procesamiento (FIFO/LIFO).	`ArrayDeque`	Colas de impresión, tareas.

2. Jerarquía y Recorridos

- `Collection` es la madre de `List` y `Set`, pero **NO** de `Map`.
- Para recorrer colecciones usamos:

  1. **For-each:** El más limpio y legible.

2. **Iterator:** El único seguro para borrar mientras recorres.
3. **Streams:** Estilo funcional (Java 8+) para filtrar y transformar.

3. Genéricos (`<T>`)

La herramienta de seguridad definitiva.

- Evita el uso de `Object` ("Cajas sorpresa").
- Garantiza en **tiempo de compilación** que los tipos son correctos (`List<String>`).
- Elimina la necesidad de hacer **Casting** manual.